Typed assembly for worst-case execution time analysis

Marcus Janum

May 12, 2026

r	::=		*registers:*	ι	::=	*instructions:*
		`r1` | $\cdots$ | `rk`				r_d`:=`v
v	::=		*operands:*			r_d`:=`$r_s + v$
		n	*integer literal*			`if` r`jump v`v
		ℓ	*label or pointer*	I	::=	*instruction sequences:*
		r	*registers*			`jump` v
						$\iota; I$

Figure 1: instructions and operands for the `tal0` language

1 Introduction

2 Typed Assembly Languages

Typed assembly languages are a group of languages (not unlike assembly) or rather just an idea that in varying degrees introduce type safety to programs written or compiled to assembly. The primary motivation for typed assembly languages is type safety and is strongly related to proof-carrying code [2, ch. 5].

In this section we will cover the canonical reason for introducing types as well as exploring levels of typing and implementations. Most of this section is based on Greg Morrissetts work whom has contributed most of the published research in this topic.

2.1 Why type safety in assembly?

Running untrusted code

2.2 TAL-0

As with PCC, it is of great interest to prove correctness of code. In modern programming languages this is done in part with a strongly typed syntax and semantics as "well-typed programs cannot 'go wrong'", thus moving some of the burden of correctness to the type system. The goal of TAL-0 (besides being a paedological tool from Morrissett to introduce readers to typed assembly) is to restrict control-flow to ensure control-flow safety of assembly code.

An example of a potential mishap — and how it may look solved — may be as the following risc-v snippet; jumping to the contents of a register, *hopefully* an address.

```
jump:
        jr      a0
```

This short example showcases how naive the machine is, as no checks need be performed, and no assurance is made, that 'a0' does indeed contain a valid code address. `tal0` solves this by special syntax and an abstract machine that only allows some evaluation; the above assembly snippet would be disallowed by the evaluation rules.

By figure 1, the previous code snippet would look like

```
jump:
    jump        r1
```

2.2.1 The abstract machine

The abstract machine presented here is in accordance to Morrissets TAL-0. The machine presented here will be extended in future sections.

2.2.2 Typing rules

$ftv(\tau)$ is the set of free variables for τ. Not sure where it looks to determine free-ness? a variable x is bound in τ if $\forall x.\tau$, but i think this may be a writing mistake? Surely it should be like $ftv(\forall x.y) = \{y\}$ while $ftv(\forall x.x) = \{\emptyset\}$

2.2.3 Soundness

The proof relies on the abstract machine not getting stuck.

3 worst-case execution time analysis

Worst-case execution time analysis is an important step in the process of developing real-time systems [3] like ssd_os. In general, and is easily examplified, not all programs are subject to execution time analysis, nor do all programs exit. The former is a consequence of the halting problem [4, ch. 13] [3].

3.1 Observed observation time

3.2 static analysis

4 ssd os

5 Possibilities

5.1 Control flow

One of the motivations behind typed assembly languages as presented earlier (TODO: REF SECTION) is control flow. TAL-0 introduces types to code to restrict where jumps are allowed to target. Control flow in regards to execution time analysis is not so interested in the validity of jumps but on the amount of jumps, i.e. loops.

5.2 Typing memory

6 Getting to a language

This section describes the process, the steps, and the considerations for designing the risc-v styled typed assembly language suited for static worst-case execution time analysis. Only a subset of the idea of the language will be fully fleshed out, and an even smaller part implemented. The "full" language requires much too complex typing that involves constraint satisfaction of predicate logic, far out of the scope of this project, unfortunately.

region memory types variables	μ		
region memory types	M	::=	$scratch \mid shared \mid \mu$
stack types variables	ρ		
stack types	S	::=	$[] \mid \rho \mid \tau :: S$
types	τ	::=	$.. \mid \tau\ array(x)$

Figure 2: Syntax for lifting types into region memory

6.1 Strays from risc-v

As with the TALs introduced above, there is little reason to distinguish between immediate instructions and register instructions, meaning that `subi` and `sub` can be treated the same. This is due to the value type that they all share in some form, defined in some way as

$$\text{value} \quad v \quad ::= \quad c \mid r$$

For code generation, these will have to be reintroduced, but this is trivial, since the value field will either be an immediate value or a register.

6.2 Stacks and arrays

This section describes how the array and stack types work with each other. From the syntax we have

Stack types can be instantiated with

$$\frac{\Phi; \Delta; (R, S) \vdash r_s : \rho \quad imm/4 = n \quad n > 0 \quad \Phi; \Delta; (R[r_d : top_0 :: \cdots :: top_n :: \rho], S) \vdash}{\Phi; \Delta; (R, S) \vdash \texttt{subi}\ r_d, r_s, imm; I} \ (::)$$

This of course lacks some expressiveness, since rarely is an extended

this immediately creates a problem, since only the risc-v stack grows downwards, and the region stacks (of which allocations happen to). However, there is a solution in the conceptualization of these stacks.

An example program that stores some values on the stack would look like the following

```
start:  [a0: int(i), a1: int(j), a2: int(k), a3: int(l)]
        subi    sp, sp, 16      ; sp: scratch top :: top :: top :: top :: 'r
        mv      a4, sp
coerce: ('m)
        [a0: int(i), a1: int(j), a2: int(k), a3: int(l), a4: 'm int array(4)]
        lw      a0, 0(a4)       ; a4: scratch int(i) :: ..
        lw      a1, 4(a4)       ; a4: scratch int(i) :: int(j) :: ..
        lw      a2, 8(a4)       ; a4: scratch int(i) :: int(j) :: int(k) ..
        lw      a3, 12(a4)      ; a4: scratch int(i) :: int(j) :: int(k) ::
            int(l) :: 'r
```

`::` is right associative, so `scratch top :: top .. = scratch(top :: top ..)`

`top` can be coerced into any type, and is used to initialize the array.

A stack $[\tau_0 :: \cdots :: \tau_n]$ can be coerced into an array $\tau\ array(\texttt{m})$ iff $m < n$ and $\tau = tau_i$ for all $0 \leq i < m$

`'m` is a type variable that will be unified to type `scratch`.

$$
\texttt{incr}\ r_d,\ r_s,\ v \quad \mapsto \quad \begin{cases} \texttt{mul}, \quad r_d, \quad v, \quad \mathit{wordsize} \\ \texttt{sub}, \quad r_d, \quad r_s, \quad \mathit{rd} \end{cases} \quad \text{iff} \quad \Phi;\Delta;R \vdash v : \text{is register}
$$

$$
\qquad\qquad\qquad\quad \begin{cases} \texttt{subi}, \quad r_d, \quad r_s, \quad v \cdot \mathit{wordsize} \end{cases} \quad \text{iff} \quad \Phi;\Delta;R \vdash v : \text{is immediate value}
$$

$$
\texttt{decr}\ r_d,\ r_s,\ v \quad \mapsto \quad \begin{cases} \texttt{mul}, \quad r_d, \quad v, \quad \mathit{wordsize} \\ \texttt{add}, \quad r_d, \quad r_s, \quad \mathit{rd} \end{cases} \quad \text{iff} \quad \Phi;\Delta;R \vdash v : \text{is register}
$$

$$
\qquad\qquad\qquad\quad \begin{cases} \texttt{addi}, \quad r_d, \quad r_s, \quad v \cdot \mathit{wordsize} \end{cases} \quad \text{iff} \quad \Phi;\Delta;R \vdash v : \text{is immediate value}
$$

Figure 3: Code generation for `incr` and `decr`

Since the stack pointer points to the stack, and the target is a specific ssd os system (link) the type of `sp` will always be `scratch`S.

Another use is working with the arena allocators. They are simple, and also work like a stack, but these grow upwards, which is opposite the call stack and therefore the typing rules would not work.

```
struct memory_region
{
    uint8_t *start;
    size_t   size;
};

struct arena_allocator
{
    struct memory_region region;
    size_t               allocated;
};
```

6.2.1 Unitialized values

Since the stack `incr` instruction essentially creates garbage values, we may want to type it, and introduce a notion of instantiated and uninstantiated types [1].

Essentially, the regions type — while in C is just a pointer — is a lifted stack type containing the whole region. When memory is allocated in an arena, the type of the returned memory is a coerced slice of the arena. If an arena has region of type $\texttt{top}_0\texttt{::}\cdots\texttt{::top}_n\texttt{::[]}$, then an offset of k words gives the stack $\texttt{top}_k\texttt{::}\cdots\texttt{::top}_n\texttt{::[]}$, which can be coerced into an array of fitting type and size.

Since the language — like other TALs — only work on word sized types but operations on memory in risc-v (like moving the stack pointer) is not aligned, we want to constrain the syntax to only allow word-size operations. Only stacks can change size (through arithmetic operations on a stack pointer) so it suffices to introduce instructions `incr` and `decr`. They may be best explained through their code generation, shown in figure 3

6.3 Types

As discussed, we want to lift types into memory regions as in figure 2. The evaluation will be as in TAL-0

6.4 Abstract machine

Unlike other TAL abstract machines, I am omitting the stack, since it will be used as a normal register (sp).

6.5 Dependent types

Because of the restrictions on the implementation domain, we can restrict the dependent types to contain fewer index propositions.

```
L0:     {n: int | n > 0}
        [a0: int(n)]
        muli    a1, a0, 4       ; a1: n * 4
        sub     sp, sp, a1      ; sp: top_0 :: ... :: top_n :: sp
        mv      a1, a0
        mv      a0, sp
L1:     ('m)
        {n: int | n > 0}
        [a0: 'm int array(n), a1: int(n)]
```

Is a valid program. The two n s are locally scoped, so they share no information, however they should represent the same value in this case. Since n is not known, only that it is a natural non-zero number, it is not known how many words are pushed to the stack, therefore its type depends on n.

However, since the lack of control flow, loops do not exist and the language turns novel. Then there is no way to store n words, and therefore no reason to be able to allocate a variable amount of words. The language reduces to constant expressions. In future work, reintroducing control flow is a must, and as discussed previously (TODO: HOPEFULLY) dependent types can be used to infer some loop iterations. For this, we would sorely miss type variables in index expressions.

this reduces stack type rules to

$$\frac{}{\Phi;\ \Delta;}\ (\text{sub})$$

6.6 Coercion

6.7 Syntax

7 Implementation (wrong title)

7.1 Domain

Only a small subset of what has been discussed will be implemented, namely typing memory region types for the scratchpad system model. The typing rules will ensure that it is known before runtime of which memory region each load/store operation writes or reads from. Since the system works without cache (scratch region effectively taking its place) all IO has a set cost,

type variables	α		
region memory types variables	μ		
region memory types	M	::=	$\mathit{scratch} \mid \mathit{shared} \mid \mu$
stack types variables	ρ		
stack types	S	::=	$[] \mid \rho \mid \tau :: S$
Collection types	C	::=	$M\ \tau\ \mathit{array}(x) \mid M\ S$
types	τ	::=	$\alpha \mid \mathit{int} \mid C$
type variable contexts	Δ	::=	$\cdot_{\mathrm{tv}} \mid \Delta, \mu \mid \Delta, \rho \mid \Delta, \alpha$

Figure 4: Syntax for a typed assembly language

and as do the arithmetic instructions. The problem then boils down to essentially summing up the cost of each instruction.

The solution is somewhat novel, since there is a painful lack of control-flow. In the theory description some outline of how this may be thought of and implemented is discussed. The biggest constraint and factor of deciding that control-flow is out of scope is due to the solving process (predicate logic SAT solving).

References

[1] Greg Morrisett. stack-based typed assembly language.

[2] Benjamin C. Price. Advanced topics in types and programming languages.

[3] Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, Frank Mueller, Isabelle Puaut, Peter Puschner, Jan Staschulat, and Per Stenström. The worst-case execution-time problem—overview of methods and survey of tools. *ACM Trans. Embed. Comput. Syst.*, 7(3), May 2008.

[4] Torben Ægidius Mogensen. Programming language design and implementation.